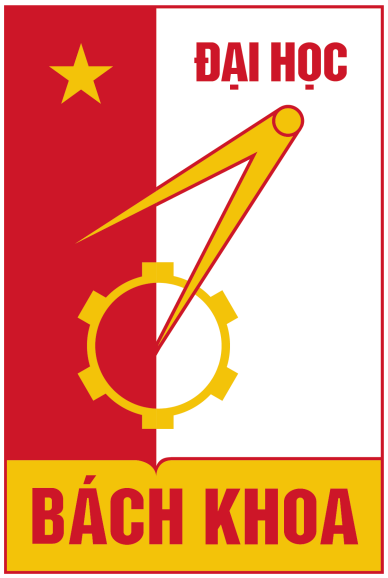
ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN–TIN



BÁO CÁO CUỐI KỲ
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT
CHỦ ĐỀ: XÂY DỰNG HỆ THỐNG QUẢN LÝ KHÁM BỆNH

Giảng viên hướng dẫn:	**TS. Nguyễn Huy Trường**
Mã học phần:	**MI3060**
Mã lớp học:	**163604**
Nhóm:	**07**
Sinh viên thực hiện:	**Nguyễn Hữu Nhật Anh – 20237412** **Nguyễn Ngô Khải – 20237412** **Trần Văn Thịnh – 20237486**

Hà Nội – 2025

Mục lục

Lời mở đầu **2**

1. Bài toán thực tế **3**

2. Các cấu trúc dữ liệu và giải thuật **4**

3. Thiết kế phân chia chương trình **13**

4. Kết quả thử nghiệm và đánh giá hiệu năng **15**

Phụ lục **29**

Tài liệu tham khảo **44**

Lời mở đầu

Cấu trúc dữ liệu và Giải thuật (Data Structures and Algorithms – DSA) là một trong những học phần nền tảng và quan trọng của Khoa học máy tính. Môn học giữ vai trò then chốt trong việc trang bị cho người học kiến thức và kỹ năng để hiểu rõ cách tổ chức dữ liệu, cách vận hành thuật toán và chương trình. Không chỉ giúp viết chương trình chạy đúng, DSA còn hướng tới việc xây dựng giải pháp hiệu quả, tối ưu thời gian và tài nguyên. Việc nắm vững DSA cũng góp phần phát triển tư duy logic, tư duy hệ thống và năng lực giải quyết vấn đề.

Thông qua môn học, sinh viên được tiếp cận các cấu trúc dữ liệu cơ bản như mảng, danh sách liên kết, ngăn xếp, hàng đợi, cây, đồ thị,... cùng với các giải thuật tiêu biểu để xử lý bài toán hiệu quả. Từ đó, người học dần hình thành phong cách lập trình chuyên nghiệp, biết thiết kế chương trình rõ ràng, có tổ chức và dễ bảo trì.

Với project cuối kỳ — *"Xây dựng Hệ thống quản lý khám bệnh"* — nhóm vận dụng các kiến thức đã học để xây dựng chương trình bằng ngôn ngữ Python. Báo cáo trình bày bài toán thực tế, các cấu trúc dữ liệu và giải thuật sử dụng, cấu trúc chương trình, kết quả thử nghiệm và đánh giá hiệu năng, kèm phụ lục mã nguồn hàm `main` và đặc tả các hàm chức năng.

Nhóm xin chân thành cảm ơn thầy Nguyễn Huy Trường đã giảng dạy nhiệt tình và chia sẻ nhiều kiến thức quý báu trong suốt học phần. Do hạn chế về thời gian và kinh nghiệm, nhóm mong nhận được những nhận xét, góp ý để báo cáo được hoàn thiện hơn.

Hà Nội, ngày 05 tháng 01 năm 2026

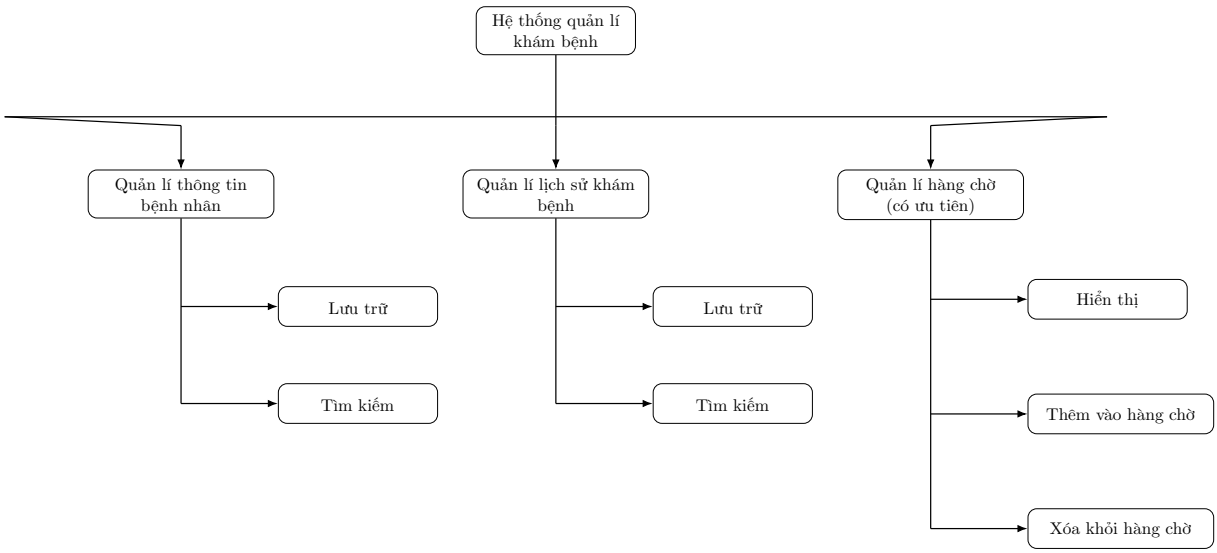
Nhóm báo cáo số 07
Nguyễn Hữu Nhật Anh – 20237412
Nguyễn Ngô Khải – 20237447
Trần Văn Thịnh – 20237486

1. Bài toán thực tế

Bài toán mà chương trình giải quyết đó chính là xây dựng một hệ thống giúp quản lí việc khám bệnh tại bệnh viện.

Trên thực tế để xây dựng một chương trình quản lí việc khám bệnh đầy đủ cần nhiều quy trình hơn nữa, nhưng do giới hạn về thời gian, thiếu kinh nghiệm và kiến thức trong việc phân tích thiết kế hệ thống phức tạp với nhiều quy trình, chương trình được thiết kế với những chức năng cần thiết nhất. Giả sử áp dụng cho một bệnh viện tư nhân quy mô nhỏ, không có nhiều phòng khám khác nhau mà chỉ có một phòng khám đa khoa duy nhất để khám cho tất cả bệnh nhân.

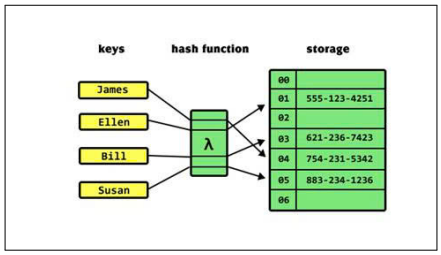
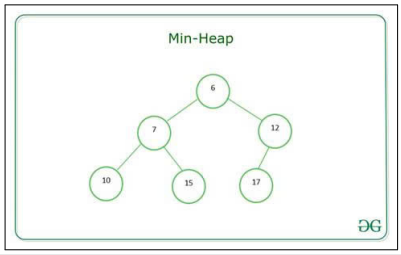
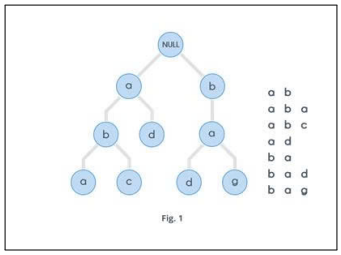
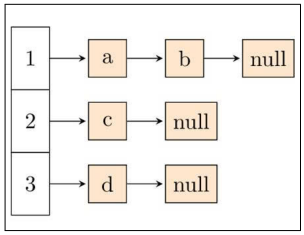
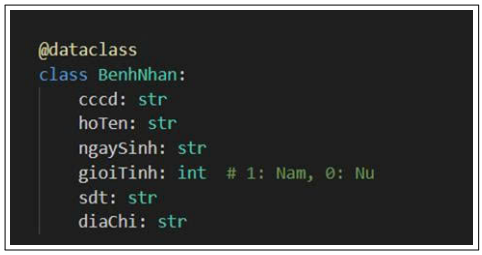
Chương trình giải quyết các nhiệm vụ quản lí thông tin bệnh nhân, quản lí lịch sử khám bệnh và quản lí hàng chờ khám bệnh tại bệnh viện.



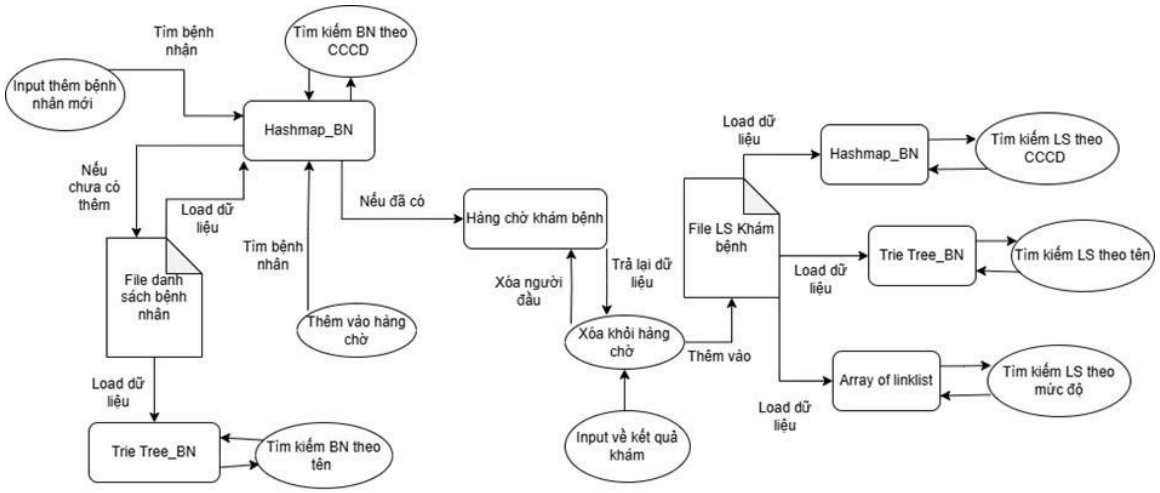
Hình 1: Sơ đồ chức năng hệ thống quản lí khám bệnh

Chương trình trong báo cáo được xây dựng nhằm mục đích ứng dụng các cấu trúc dữ liệu và giải thuật được học vào bài toán thực tế, hướng tới chiều sâu đánh giá, so sánh nhiều thuật toán với nhau để nhằm đưa ra giải pháp tối ưu nhất. Chương trình được xây dựng bằng ngôn ngữ Python.

2. Các cấu trúc dữ liệu và giải thuật

Tên cấu trúc dữ liệu	Thực hiện chức năng
Hashtable – Hàm băm 	Tìm kiếm bệnh nhân và lịch sử khám bệnh theo số Căn cước công dân.
Min-Heap 	Cài đặt hàng chờ khám bệnh có ưu tiên; hỗ trợ thêm và xoá bệnh nhân ra khỏi hàng chờ khám bệnh.
Inverted Index 	Tìm kiếm bệnh nhân và lịch sử khám bệnh theo tên và các thành phần trong tên.
Array of Linkedlists 	Tìm kiếm danh sách bệnh nhân theo các mức độ khám bệnh.
Các kiểu dữ liệu định nghĩa bằng Struct 	Lưu trữ, trao đổi thông tin của các đối tượng cần quản lí trong bài toán đã nêu ra.

Bảng 1: Các cấu trúc dữ liệu sử dụng trong chương trình



Hình 2: Sơ đồ mô tả tổng quát luồng dữ liệu của chương trình

2.1. Các kiểu dữ liệu trong chương trình

Trong chương trình, dữ liệu được mô tả dưới dạng các lớp sử dụng **@dataclass** để quản lí thông tin theo từng "thực thể" nghiệp vụ. Các kiểu dữ liệu chính gồm: **BenhNhan**, **PhieuKham**, **LichSuKham** và **MenuItem**.

2.1.1. Lớp `BenhNhan`

Chức năng: Lưu trữ toàn bộ thông tin cơ bản của một bệnh nhân, phục vụ cho các thao tác thêm mới, tra cứu, in thông tin và tạo phiếu khám.

Thuộc tính dữ liệu:

- `cccd: str` – số CCCD của bệnh nhân (dùng làm khóa nhận diện).
- `hoTen: str` – họ tên bệnh nhân.
- `ngaySinh: str` – ngày sinh (định dạng `DD/MM/YYYY`).
- `gioiTinh: int` – giới tính (1: Nam, 0: Nữ).
- `sdt: str` – số điện thoại.
- `diaChi: str` – địa chỉ.

Kiểm tra hợp lệ đầu vào:

- CCCD: đúng 12 ký tự số.
- SĐT: đúng 10 ký tự số.
- Họ tên: độ dài từ 4 đến 50 ký tự (sau khi loại khoảng trắng thừa).
- Địa chỉ: độ dài từ 3 đến 50 ký tự.
- Ngày sinh: đúng định dạng `DD/MM/YYYY` và kiểm tra miền giá trị ngày/tháng/năm.

Cách sử dụng trong chương trình:

- Khi thêm bệnh nhân, hệ thống kiểm tra trùng CCCD bằng tra cứu trước, sau đó tạo đối tượng `BenhNhan`.
- Dữ liệu bệnh nhân được ghi vào file, định dạng 6 trường phân tách bởi ký tự `|`:

  `cccd|hoTen|ngaySinh|gioiTinh|sdt|diaChi`

- Sau khi ghi file, dữ liệu được cập nhật vào cấu trúc tra cứu (hash) và tìm kiếm theo tên (trie).

2.1.2. Lớp `PhieuKham`

Chức năng: Lưu thông tin một lượt đăng ký khám của bệnh nhân và là phần tử được đưa vào **danh sách chờ khám** (heap ưu tiên).

Thuộc tính dữ liệu:

- `bn: BenhNhan` – thông tin bệnh nhân (dạng đối tượng).
- `mucUuTien: int` – mức độ ưu tiên (1: cấp cứu, 2: ưu tiên, 3: thường).
- `moTa: str` – mô tả/loại bệnh khi đăng ký (giới hạn độ dài).
- `sttKham: int` – số thứ tự khám (chỉ áp dụng cho mức 3).
- `arrival: int` – thứ tự đến (hỗ trợ phá hòa và đảm bảo FIFO trong nhóm 1/2).
- `thoiGianDangKy: str` – thời điểm đăng ký theo dạng `DD/MM/YYYY HH:MM:SS`.

Cách sử dụng trong chương trình:

- Khi thêm vào hàng chờ khám, chương trình:
  1. Nhập CCCD và tra cứu lấy bệnh nhân.
  2. Nhập mức ưu tiên và mô tả.
  3. Sinh thời gian đăng ký, cập nhật bộ đếm `arrival` và `sttKham`.
  4. Tạo `PhieuKham` và đưa vào heap.

2.1.3. Lớp `LichSuKham`

Chức năng: Lưu một bản ghi lịch sử khám sau khi bệnh nhân đã được gọi vào khám xong; dùng cho các chức năng tra cứu lịch sử theo CCCD/tên/mức độ.

Thuộc tính dữ liệu:

- `cccd: str` – CCCD bệnh nhân.
- `hoTen: str` – họ tên.

- `ngaySinh: str` – ngày sinh.
- `mucDo: int` – mức độ (1/2/3).
- `ngayKham: str` – ngày khám (`DD/MM/YYYY`).
- `moTa: str` – mô tả/ghi chú kết quả khám.

Cách sử dụng trong chương trình:

- Khi gọi khám, sau khi nhập `ngayKham` và `kết quả khám`, chương trình ghi một dòng lịch sử vào file theo đúng 6 trường:

  `cccd|hoTen|ngaySinh|mucUuTien|ngayKham|moTa`

- Đồng thời tạo đối tượng `LichSuKham` và thêm vào bộ quản lí lịch sử.

2.1.4. Lớp `MenuItem`

Chức năng: Biểu diễn một lựa chọn trong menu chương trình, gồm mã lựa chọn, nội dung hiển thị và hàm xử lý tương ứng.

Thuộc tính dữ liệu:

- `id: int` – mã lựa chọn.
- `text: str` – nội dung hiển thị.
- `action: Optional[Callable[[], None]]` – hàm sẽ được gọi khi người dùng chọn mục đó (có thể `None` để thoát).

2.2. Hashtable

2.2.1. Mô tả

Trong chương trình, **Hashtable** được sử dụng để lưu trữ và tra cứu dữ liệu theo cơ chế ánh xạ `key` $\rightarrow$ `value`. Việc cài đặt Hashtable được thực hiện trực tiếp bằng cấu trúc `dict` của Python, trong đó:

- `key` là chuỗi định danh (ví dụ: `CCCD`);
- `value` là đối tượng dữ liệu tương ứng (ví dụ: `BenhNhan` hoặc danh sách `LichSuKham`).

2.2.2. Cài đặt trong chương trình

(1) Hashtable quản lý bệnh nhân theo CCCD

Chương trình xây dựng lớp `BenhNhanHash` để quản lý bệnh nhân theo CCCD. Cấu trúc lưu trữ chính là:

`table: dict[str, BenhNhan]`

Các thao tác chính:

- **Thêm bệnh nhân:** `insert(bn)`
  Gán trực tiếp `table[bn.cccd] = bn`. Nếu CCCD đã tồn tại thì dữ liệu sẽ bị ghi đè (vì vậy chương trình luôn kiểm tra trùng trước khi thêm mới).
- **Tìm bệnh nhân theo CCCD:** `search(cccd)`
  Dùng `table.get(cccd)`; nếu không tồn tại thì trả về `None`.
- **Duyệt toàn bộ bệnh nhân:** `values()`
  Trả về `table.values()` để phục vụ nạp dữ liệu sang cấu trúc khác (ví dụ Trie).
- **Nạp dữ liệu từ file:** `load_from_file(filename)`
  Đọc file `BENHNHAN.TXT`, tách 6 trường theo ký tự `|`, tạo đối tượng `BenhNhan` và đưa vào hash bằng `insert()`.

(2) Hashtable quản lý lịch sử khám theo CCCD

Trong quản lý lịch sử, chương trình lưu lịch sử theo CCCD bằng cấu trúc:

```
by_cccd: dict[str, list[LichSuKham]]
```

Mỗi `CCCD` ánh xạ tới **một danh sách** các lần khám. Khi thêm bản ghi lịch sử mới, chương trình dùng:

```
by_cccd.setdefault(cccd, []).append(ls)
```

Nhờ đó, các lần khám của cùng một bệnh nhân được gom lại theo đúng CCCD để tra cứu nhanh.

2.2.3. Ứng dụng trong các chức năng

Hashtable được sử dụng ở nhiều vị trí quan trọng trong chương trình:

- **Kiểm tra trùng CCCD khi thêm bệnh nhân:**
  Khi nhập CCCD, hệ thống tra cứu nhanh trong hash; nếu đã tồn tại thì yêu cầu nhập lại, đảm bảo CCCD là duy nhất.
- **Tìm bệnh nhân theo CCCD:**
  Phục vụ chức năng tra cứu bệnh nhân và đặc biệt là bước *lấy thông tin bệnh nhân* trước khi tạo `PhieuKham` để đưa vào hàng chờ khám.
- **Tra cứu lịch sử khám theo CCCD:**
  Khi cần xem lịch sử theo CCCD, chương trình lấy trực tiếp danh sách bản ghi từ `by_cccd` và in ra toàn bộ các lần khám.

2.3. Min – Heap

2.3.1. Mô tả

Trong chương trình, **Min-Heap** được dùng để cài đặt **hàng đợi ưu tiên** phục vụ chức năng *quản lý hàng chờ khám*. Đặc điểm của Min-Heap là phần tử có **giá trị khóa nhỏ nhất** sẽ luôn nằm ở đỉnh, do đó thao tác lấy phần tử ưu tiên nhất diễn ra nhanh.

Python hỗ trợ Min-Heap thông qua thư viện `heapq`, cung cấp các thao tác:

- `heappush(heap, item)`: thêm một phần tử vào heap;
- `heappop(heap)`: lấy và loại bỏ phần tử nhỏ nhất khỏi heap.

Trong hệ thống khám bệnh, Min-Heap đảm bảo bệnh nhân được gọi khám theo đúng quy tắc ưu tiên và thứ tự đến.

2.3.2. Cài đặt trong chương trình

Chương trình xây dựng lớp `KhamManager` để quản lý hàng chờ khám. Dữ liệu hàng chờ được lưu bằng:

```
heap: list[tuple[key, PhieuKham]]
```

Trong đó mỗi phần tử là một cặp:

- `key`: bộ khóa sắp xếp (tuple) dùng để so sánh trong heap;
- `PhieuKham`: thông tin phiếu khám tương ứng.

Thiết kế khóa ưu tiên `key` Để đảm bảo đúng thứ tự gọi khám, chương trình tạo khóa bằng hàm `_make_key(pk)`. Khóa được thiết kế theo dạng tuple để sắp xếp theo nhiều tiêu chí:

- **Mức ưu tiên**: quy đổi sao cho mức 1 nhỏ nhất (ưu tiên cao nhất), sau đó đến mức 2 và mức 3.
- **Nhóm ưu tiên**: tách nhóm {1,2} và nhóm {3} để xử lý khác nhau.
- **Tiêu chí thứ ba**:
  - Với mức 1/2: dùng `arrival` để đảm bảo FIFO theo thứ tự đến trong nhóm ưu tiên.
  - Với mức 3: dùng `sttKham` để khám theo số thứ tự khám thường.
- **Tiêu chí phá hòa**: kèm thêm `arrival` ở cuối để đảm bảo thứ tự ổn định khi các tiêu chí trước bằng nhau.

Nhờ đó, heap luôn pop ra đúng bệnh nhân cần khám trước theo quy tắc của hệ thống.

2.3.3. Ứng dụng trong các chức năng

Min-Heap xuất hiện trực tiếp trong các chức năng chính sau:

- **Thêm vào hàng chờ khám:**
  1. Nhập CCCD và tra cứu bệnh nhân (bằng hashtable).
  2. Nhập mức ưu tiên và mô tả bệnh.
  3. Tạo đối tượng `PhieuKham` (cập nhật `arrival` và `sttKham`).
  4. Tạo `key` và dùng `heappush` để đưa vào heap.
- **Gọi khám (lấy bệnh nhân tiếp theo):**
  1. Lấy phần tử ưu tiên nhất bằng `heappop`.
  2. Hiển thị thông tin bệnh nhân và phiếu khám.
  3. Nhập kết quả khám (ngày khám, mô tả).
  4. Ghi lịch sử khám và cập nhật cấu trúc quản lý lịch sử.
- **Xem danh sách đang chờ:**
  Chương trình có thể duyệt danh sách heap (hoặc hiển thị bản sao đã sắp thứ tự) để người dùng quan sát trạng thái hàng chờ hiện tại.

2.4. Cây tiền tố – Trie Tree

2.4.1. Mô tả

Trong chương trình, **Trie Tree** được sử dụng để hỗ trợ **tìm kiếm theo từ khóa trong tên** một cách nhanh và hiệu quả. Trie là cấu trúc dạng cây, trong đó mỗi cạnh tương ứng với một ký tự. Khi duyệt từ gốc theo các ký tự của một từ khóa, ta sẽ đi tới nút biểu diễn từ khóa đó; từ nút này có thể lấy ra danh sách các bản ghi đã gắn với từ khóa.

Chương trình dùng Trie để:

- tìm bệnh nhân theo từ khóa trong `hoTen`;
- tìm lịch sử khám theo từ khóa trong `hoTen`.

2.4.2. Cài đặt trong chương trình

Trie được cài đặt trong file `trie_ds.py` với hai lớp chính:

- `TrieNode`: mỗi nút chứa
  - `children: dict[str, TrieNode]` – ánh xạ ký tự → nút con;
  - `records: list[object]` – danh sách các bản ghi được gắn tại nút này.
- `Trie`: cung cấp các thao tác chèn và tìm kiếm.

Cách chèn dữ liệu Khi chèn một bản ghi, chương trình lấy chuỗi tên, chuẩn hóa về chữ thường và tách thành các từ theo dấu cách. Với mỗi từ (token), Trie sẽ duyệt theo từng ký tự để tạo nhánh nếu chưa tồn tại, sau đó gắn bản ghi vào `records` tại nút cuối của token. Cách làm này giúp việc tìm theo từ khóa là tìm theo đúng một token bất kỳ trong tên.

Loại trùng kết quả Trong thao tác tìm kiếm, do một người có thể được gắn ở nhiều token, chương trình có bước loại trùng dựa trên `id(record)` để đảm bảo mỗi bản ghi chỉ xuất hiện một lần trong kết quả.

2.4.3. Ứng dụng trong các chức năng

Trie được sử dụng trong các chức năng sau:

- **Tìm bệnh nhân theo tên:**
  Người dùng nhập một từ khóa (ví dụ một phần họ/tên). Hệ thống gọi hàm tìm kiếm trong Trie để lấy danh sách các bệnh nhân có chứa từ khóa đó trong `hoTen`, sau đó in ra thông tin bệnh nhân.
- **Tìm lịch sử khám theo tên:**
  Tương tự, người dùng nhập từ khóa; chương trình tra trong Trie của lịch sử để lấy danh sách các bản ghi `LichSuKham` có tên chứa từ khóa và in ra danh sách lịch sử tương ứng.

2.5. Danh sách liên kết – Array of Linklists

2.5.1. Mô tả

Trong chương trình, dữ liệu lịch sử khám được phân nhóm theo **mức độ** (1/2/3) để hỗ trợ truy vấn nhanh theo từng nhóm. Cách tổ chức này được triển khai bằng một **mảng các danh sách** (array of lists), trong đó mỗi phần tử của mảng là một danh sách chứa các bản ghi lịch sử thuộc cùng một mức độ.

Cấu trúc này giúp:

- truy cập trực tiếp vào nhóm mức độ cần xem;
- giảm chi phí lọc dữ liệu so với cách duyệt toàn bộ lịch sử rồi kiểm tra điều kiện.

2.5.2. Cài đặt trong chương trình

Trong lớp `LichSuManager`, chương trình khởi tạo cấu trúc:

```
by_mucdo: list[list[LichSuKham]] = [[], [], []]
```

Ý nghĩa từng vị trí:

- `by_mucdo[0]`: lưu lịch sử khám mức 1;

- `by_mucdo[1]`: lưu lịch sử khám mức 2;
- `by_mucdo[2]`: lưu lịch sử khám mức 3.

Khi thêm một bản ghi lịch sử `ls`, chương trình đưa bản ghi vào đúng danh sách theo công thức chỉ số:

```
idx = ls.mucDo - 1
```

và thực hiện:

```
by_mucdo[idx].append(ls)
```

2.5.3. Ứng dụng trong các chức năng

Cấu trúc "mảng các danh sách" được dùng trực tiếp cho chức năng **tìm lịch sử khám theo mức độ**:

- Người dùng nhập mức độ cần xem (1/2/3).
- Hệ thống truy cập đúng danh sách `by_mucdo[mucDo-1]` và in toàn bộ các bản ghi trong danh sách đó.

Bên cạnh cách tối ưu trên, chương trình còn có chức năng tìm kiếm theo kiểu "thông thường" bằng cách **đọc file và duyệt tuyến tính**, nhằm minh họa sự khác biệt về cách tổ chức dữ liệu.

3. Thiết kế phân chia chương trình

3.1 Thiết kế các module chức năng

Trong hệ thống quản lý khám bệnh, việc tổ chức mã nguồn thành các **module độc lập** là yếu tố then chốt để đảm bảo chương trình dễ phát triển, dễ bảo trì và mở rộng. Với Python, chương trình được chia theo từng nhóm chức năng rõ ràng (quản lý bệnh nhân, hàng chờ khám, lịch sử khám, giao diện menu, luồng chính), nhờ đó giảm phụ thuộc chéo giữa các phần, đồng thời tận dụng tốt các cấu trúc dữ liệu có sẵn và thư viện chuẩn để tối ưu hiệu năng.

3.2. Mục tiêu thiết kế phân chia chương trình

1. **Tăng tính tổ chức và dễ đọc:** Chia chương trình thành các file/module Python riêng như `benhnhan.py`, `kham.py`, `lichsu2.py`, `menu.py`, `main.py`... giúp mã nguồn rõ ràng, dễ theo dõi. Mỗi module đảm nhiệm một nhiệm vụ cụ thể, tránh tình trạng "gom tất cả vào một file", đặc biệt hữu ích khi xử lý các chức năng lớn như quản lý bệnh nhân, hàng chờ khám và lịch sử khám.

2. **Tăng khả năng tái sử dụng:** Một số module có thể tái sử dụng cho bài toán tương tự:

    - `menu.py` cung cấp khung menu điều khiển chương trình theo lựa chọn người dùng;
    - `trie_ds.py` triển khai Trie để tìm kiếm theo từ khóa tên;
    - `benhnhan_hash.py` đóng vai trò lớp quản lý tra cứu nhanh theo CCCD.

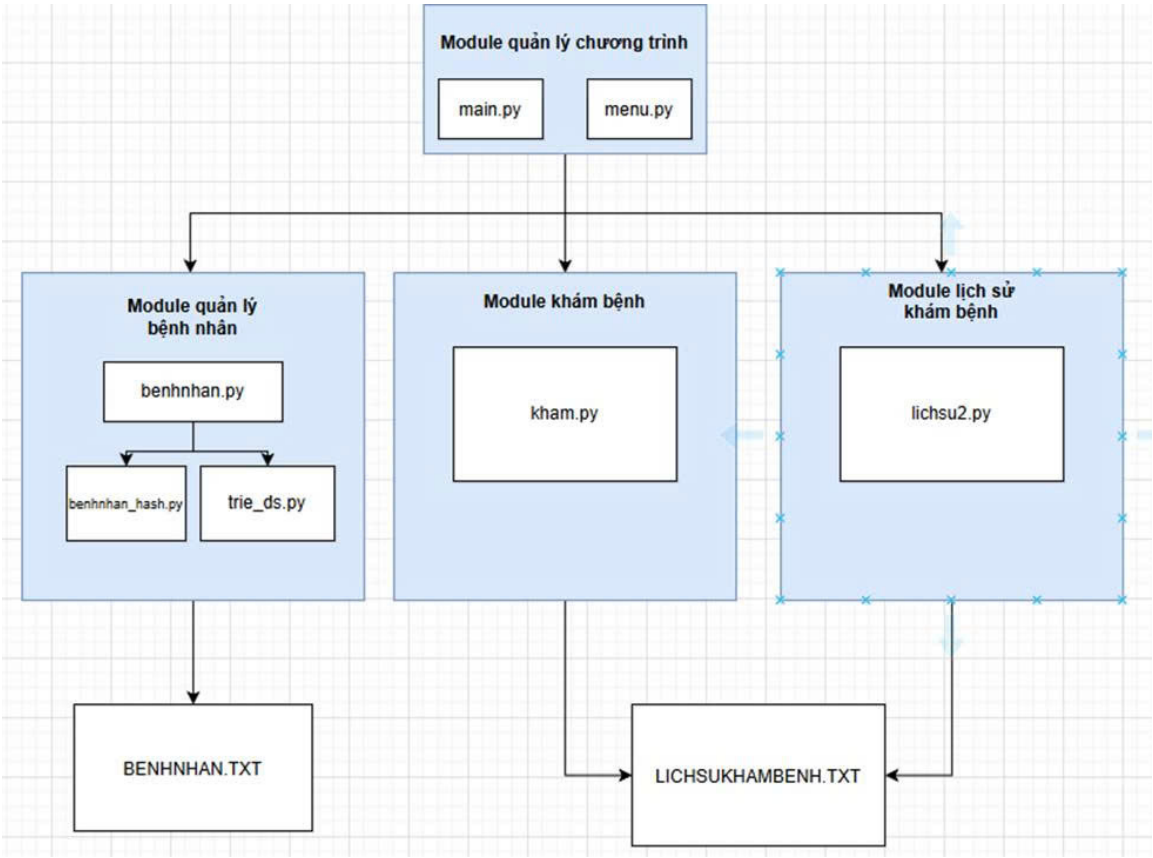
    Các module này có thể dùng lại khi xây dựng các hệ thống quản lý khác mà không cần viết lại từ đầu.

3. **Tối ưu hóa hiệu suất kỹ thuật:** Thiết kế module hóa giúp **cắm đúng cấu trúc dữ liệu vào đúng bài toán**, từ đó tối ưu thời gian xử lý:

    - Tra cứu bệnh nhân theo CCCD sử dụng bảng băm (`dict`) trong `benhnhan_hash.py`, cho thời gian tìm kiếm trung bình .
    - Tìm kiếm bệnh nhân/lịch sử theo tên sử dụng **Trie** trong `trie_ds.py`, cho thời gian tìm theo độ dài từ khóa.
    - Quản lý hàng chờ khám sử dụng **Heap** (`heapq`) trong `kham.py`, thao tác thêm/xóa phần tử.
    - Lịch sử khám trong `lichsu2.py` được tổ chức theo nhiều chỉ mục (`by_cccd`, `trie`, `by_mucdo`) giúp việc truy vấn nhanh hơn so với cách đọc tuyến tính toàn bộ file mỗi lần tìm.

4. **Dễ bảo trì và mở rộng:** Khi cần bổ sung chức năng (ví dụ: thêm tiêu chí ưu tiên mới, thêm trường dữ liệu bệnh nhân, xuất báo cáo thống kê), ta có thể chỉnh sửa **đúng module liên quan** mà ít ảnh hưởng tới phần còn lại. Điều này làm giảm lỗi dây chuyền và giúp quá trình nâng cấp chương trình an toàn hơn.

5. **Hỗ trợ làm việc nhóm và phát triển song song:** Việc chia module theo chức năng cho phép nhiều thành viên cùng làm: một người phụ trách `benhnhan.py`, người khác làm `kham.py`, người khác tối ưu `lichsu2.py`... Sau đó tích hợp qua `main.py`. Cách tổ chức này giúp giảm xung đột khi code và tăng hiệu quả phối hợp.



Hình 3: Sơ đồ phân chia module của hệ thống quản lý bệnh nhân

Trong sơ đồ trên, chương trình được chia thành các module chính như sau:

- **Module quản lý chương trình**: điều khiển luồng chạy và xử lý menu (`main.py`, `menu.py`).
- **Module quản lý bệnh nhân**: quản lý thông tin bệnh nhân, tìm kiếm theo CCCD và tên (`benhnhan.py`, `benhnhan_hash.py`, `trie_ds.py`).
- **Module khám bệnh**: quản lý danh sách chờ khám bằng Min-Heap, xử lý ưu tiên và thời gian đăng ký (`kham.py`).
- **Module lịch sử khám bệnh**: lưu trữ và tra cứu lịch sử khám (`lichsu2.py`).

Dữ liệu của hệ thống được lưu trữ trong các file văn bản `BENHNHAN.TXT` và `LICHSUKHAMBENH.TXT`, giúp chương trình hoạt động độc lập với hệ quản trị cơ sở dữ liệu.

4. Kết quả thử nghiệm và đánh giá hiệu năng

4.1 Đánh giá hiệu năng chương trình quản lý khám bệnh (Python)

Trong phiên bản Python, chương trình được thiết kế theo hướng tối ưu trải nghiệm sử dụng và thời gian phản hồi bằng cách **nạp dữ liệu lên bộ nhớ khi khởi động** và **xây dựng các chỉ mục tra cứu** phục vụ tìm kiếm nhanh. Cụ thể:

- Dữ liệu bệnh nhân được đọc từ file và nạp vào cấu trúc tra cứu theo CCCD; đồng thời đưa thông tin tên vào Trie ngay trong quá trình khởi tạo ở `main.py`.
- Lịch sử khám được nạp và tổ chức theo nhiều hướng truy vấn trong `LichSuManager` (theo CCCD, theo tên, theo mức độ ưu tiên).
- Hàng chờ khám sử dụng `heapq` với cơ chế ưu tiên rõ ràng (ưu tiên theo mức độ, có xét thứ tự đến; mức thường có số thứ tự riêng).

Nhờ cách tổ chức trên, các thao tác tra cứu và gọi khám thường cho cảm giác **nhanh và ổn định** khi dữ liệu ở quy mô nhỏ–vừa (khoảng 1,000–10,000 bản ghi). Tuy nhiên, các thao tác có liên quan đến **đọc/ghi file** vẫn phụ thuộc vào tốc độ lưu trữ và hệ điều hành.

Bảng hiệu năng các tác vụ chính

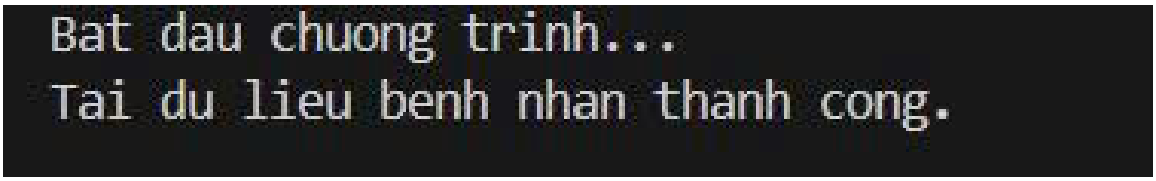
Tác vụ	Cơ chế xử lý trong chương trình	Mức phản hồi (ước lượng)
Nạp dữ liệu bệnh nhân lúc khởi động	Đọc file dữ liệu bệnh nhân, nạp vào cấu trúc tra cứu theo CCCD và đồng thời đưa tên vào Trie trong `main.py`.	Nhanh
Tìm bệnh nhân theo CCCD	Tra cứu trực tiếp bằng `dict.get()` trong `BenhNhanHash.search()`.	Tức thì
Tìm bệnh nhân theo tên (từ khóa/prefix)	Tìm theo Trie thông qua `search_keyword()` để trả về danh sách bản ghi phù hợp.	Rất nhanh
Thêm bệnh nhân mới	Kiểm tra trùng CCCD qua cấu trúc tra cứu, kiểm tra hợp lệ dữ liệu, ghi file, sau đó cập nhật Trie.	Nhanh
Thêm phiếu khám vào hàng chờ	Tạo phiếu khám và đưa vào hàng chờ bằng `heapq`.	Tức thì
Gọi khám bệnh nhân tiếp theo	Lấy bệnh nhân tiếp theo từ hàng chờ (`heappop`), nhập kết quả và ghi lịch sử khám ra file.	Nhanh → Trung bình
Nạp lịch sử khám (tối ưu) lúc khởi động	Đọc file lịch sử, tách dữ liệu đúng 6 trường bằng `split(kern.03em", 5)`, sau đó nạp vào các chỉ mục.	Nhanh
Tìm lịch sử theo CCCD (tối ưu)	Truy vấn trực tiếp trong `by_cccd` rồi hiển thị kết quả.	Rất nhanh
Tìm lịch sử theo tên (tối ưu)	Tìm theo Trie (`search_keyword`) rồi hiển thị danh sách kết quả.	Rất nhanh
Tìm lịch sử theo mức độ	Lọc theo danh sách tương ứng trong `by_mucdo[mucDo-1]` và in kết quả.	Nhanh
Tìm lịch sử (bản thường – đối chứng)	Mỗi lần tìm đều mở file và duyệt tuyến tính từng dòng để lọc dữ liệu.	Chậm hơn khi file lớn

Bảng 2: Đánh giá mức phản hồi của các tác vụ chính trong chương trình Python

4.2 Chức năng thêm bệnh nhân mới

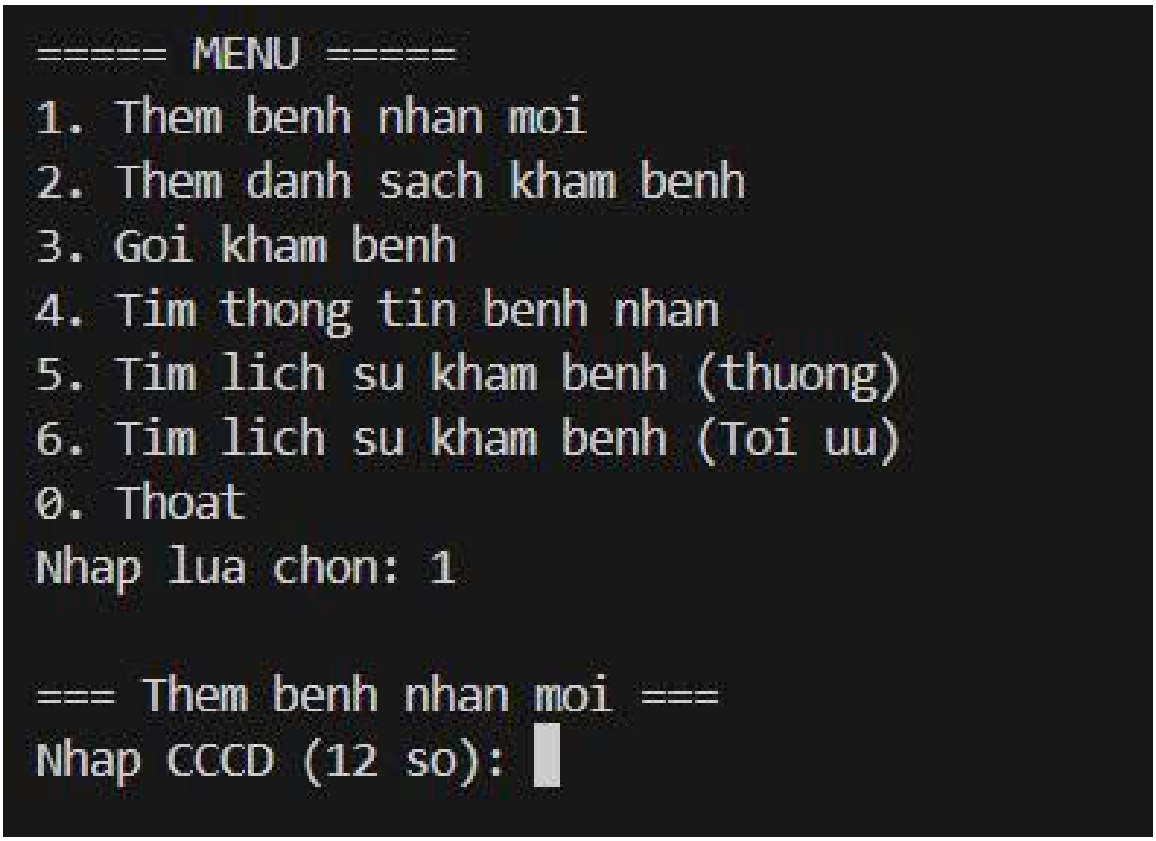
TEST 1: Chạy chương trình

- **INPUT:** Chạy chương trình.
- **OUTPUT :** Hiển thị thông báo chạy chương trình và mở file dữ liệu thành công.



TEST 2: Chọn chức năng

- **INPUT:** Nhập số "1".
- **OUTPUT :** Hiển thị các bước/thông tin yêu cầu nhập cho chức năng thêm bệnh nhân mới.



TEST 3: Nhập dữ liệu cho chức năng thêm bệnh nhân mới

- **INPUT:** Nhập bộ dữ liệu hợp lệ theo yêu cầu của chương trình.
- **OUTPUT :** Hiển thị thông báo: *"Đã thêm bệnh nhân thành công!"*.

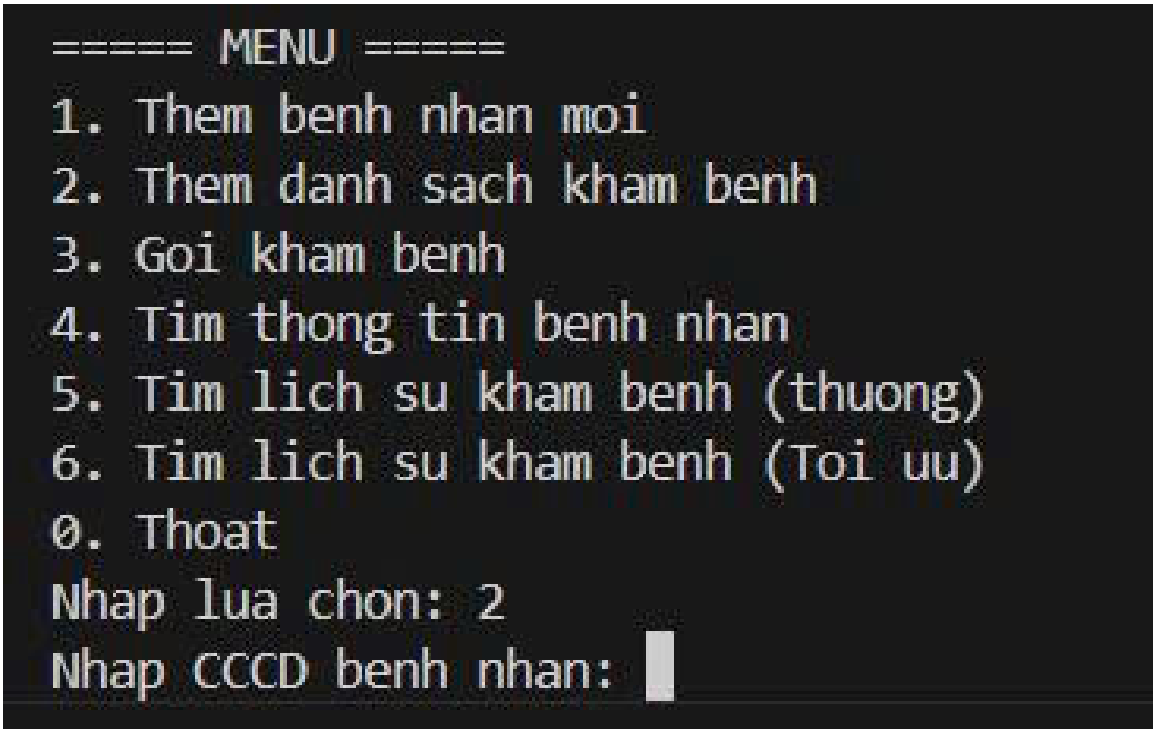
```
===== MENU =====
1. Them benh nhan moi
2. Them danh sach kham benh
3. Goi kham benh
4. Tim thong tin benh nhan
5. Tim lich su kham benh (thuong)
6. Tim lich su kham benh (Toi uu)
0. Thoat
Nhap lua chon: 1

=== Them benh nhan moi ===
Nhap CCCD (12 so): 040205029739
Nhap ho ten (4-50 ky tu): Nguyen Huu Nhat Anh
Nhap ngay sinh (DD/MM/YYYY): 20/05/2004
Nhap gioi tinh (1: Nam, 0: Nu): 1
Nhap so dien thoai (10 so): 0356967218
Nhap dia chi (3-50 ky tu): 120 Yen Lang
 Da them benh nhan thanh cong!
```

4.3 Chức năng thêm danh sách khám bệnh

TEST 4: Chọn chức năng

- **INPUT:** Nhập số "2".
- **OUTPUT :** Hiển thị các bước của chức năng thêm danh sách khám bệnh.



TEST 5: Tìm kiếm thông tin bệnh nhân cần thêm vào danh sách khám

- **INPUT:** Nhập CCCD của bệnh nhân.
- **OUTPUT :** Hiển thị thông tin bệnh nhân và các bước tiếp theo của chức năng thêm danh sách khám bệnh.

```
1. Them benh nhan moi
2. Them danh sach kham benh
3. Goi kham benh
4. Tim thong tin benh nhan
5. Tim lich su kham benh (thuong)
6. Tim lich su kham benh (Toi uu)
0. Thoat
Nhap lua chon: 2
Nhap CCCD benh nhan: 040205029639

===== Thong tin benh nhan =====
CCCD        : 040205029639
Ho Ten      : Nguyễn Ngô Khải
Ngay Sinh   : 23/01/2005
Gioi Tinh   : Nam
SDT         : 0848807582
Dia Chi     : ha noi
================================
Nhap muc do uu tien (1: cap cuu, 2: uu tien, 3: thuong):
```

TEST 6: Nhập dữ liệu cho chức năng thêm danh sách khám bệnh

- **INPUT:** Nhập bộ dữ liệu hợp lệ (ví dụ: mức độ ưu tiên, mô tả triệu chứng, . . .).
- **OUTPUT :** Hiển thị thông báo: *"Đã thêm vào danh sách cho khám"*.
- **Chú ý:** Thực hiện tương tự với các bệnh nhân khác.

```
Nhap lua chon: 2
Nhap CCCD benh nhan: 040205029639

===== Thong tin benh nhan =====
CCCD        : 040205029639
Ho Ten      : Nguyễn Ngô Khải
Ngay Sinh   : 23/01/2005
Gioi Tinh   : Nam
SDT         : 0848807582
Dia Chi     : ha noi
================================
Nhap muc do uu tien (1: cap cuu, 2: uu tien, 3: thuong): 1
Nhap mo ta/loai benh (<50 ky tu): Cap cuu noi soi da day
 Da them vao danh sach cho kham.
 (CAP CUU) Thoi gian dang ky: 05/01/2026 09:12:50
```

4.4 Chức năng gọi khám bệnh

TEST 7: Chọn chức năng

- **INPUT:** Nhập số "3".
- **OUTPUT :** Hiển thị danh sách chờ khám.

```
===== MENU =====
1. Them benh nhan moi
2. Them danh sach kham benh
3. Goi kham benh
4. Tim thong tin benh nhan
5. Tim lich su kham benh (thuong)
6. Tim lich su kham benh (Toi uu)
0. Thoat
Nhap lua chon: 3

Danh sach cho kham:
#1 [09:12:50] (UU TIEN 1) - Nguyễn Ngô Khải - 23/01/2005
```

TEST 8: Lấy bệnh nhân ra khỏi hàng chờ

- **INPUT:** Nhập ngày khám và kết quả khám bệnh theo yêu cầu chương trình.
- **OUTPUT :** Hiển thị thông báo: *"Đã lưu vào lịch sử khám bệnh"*.

```
>>> Dang goi benh nhan #1:

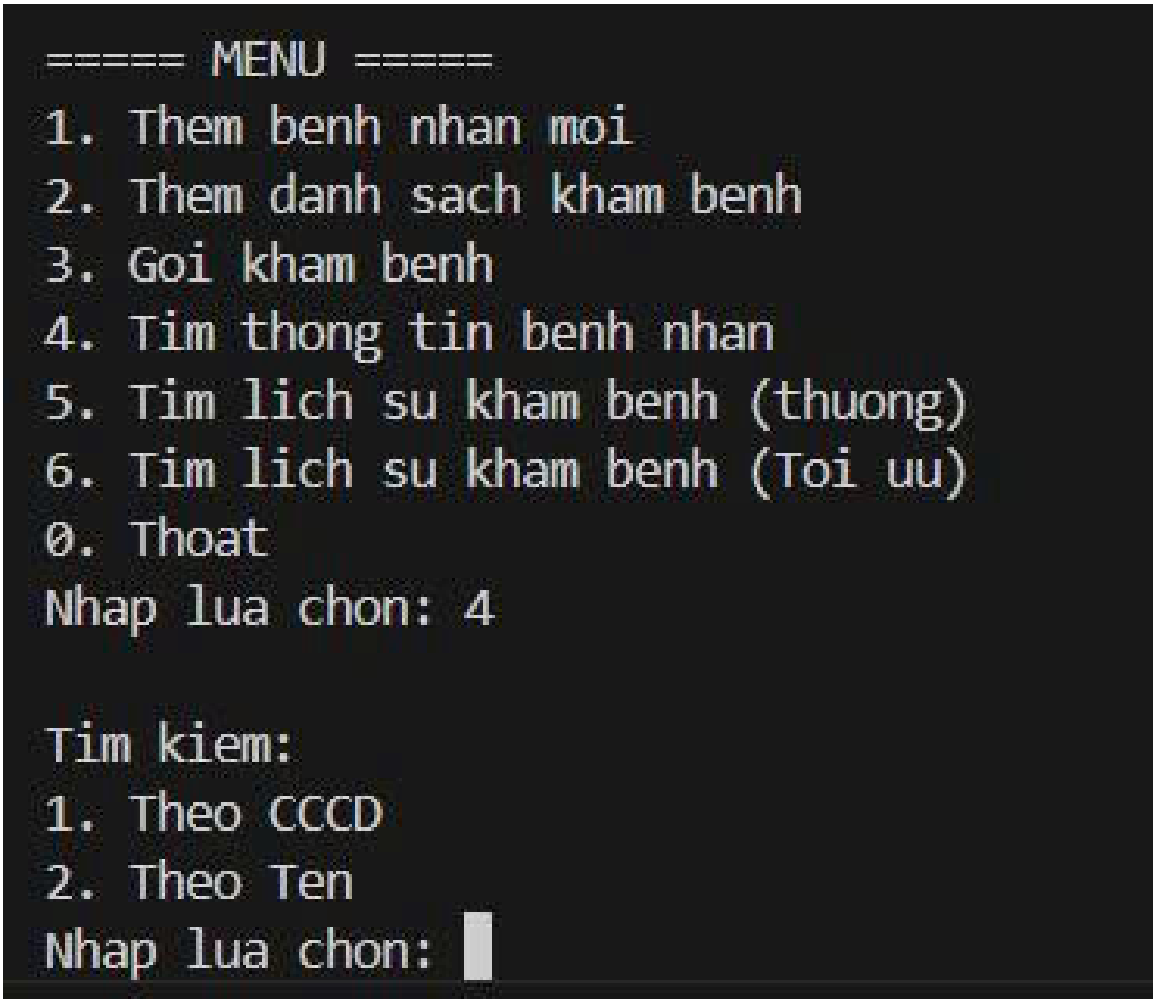
===== Thong tin benh nhan =====
CCCD        : 040205029639
Ho Ten      : Nguyễn Ngô Khải
Ngay Sinh   : 23/01/2005
Gioi Tinh   : Nam
SDT         : 0848807582
Dia Chi     : ha noi
================================
* CAP CUU - Thoi gian dang ky: 05/01/2026 09:12:50 *

Khi benh nhan kham xong an Enter...
Nhap ngay kham (DD/MM/YYYY): 01/04/2026
Nhap ket qua kham benh (<50): Khoi benh
 Da luu vao lich su kham benh.
```

4.5 Chức năng tìm kiếm thông tin bệnh nhân

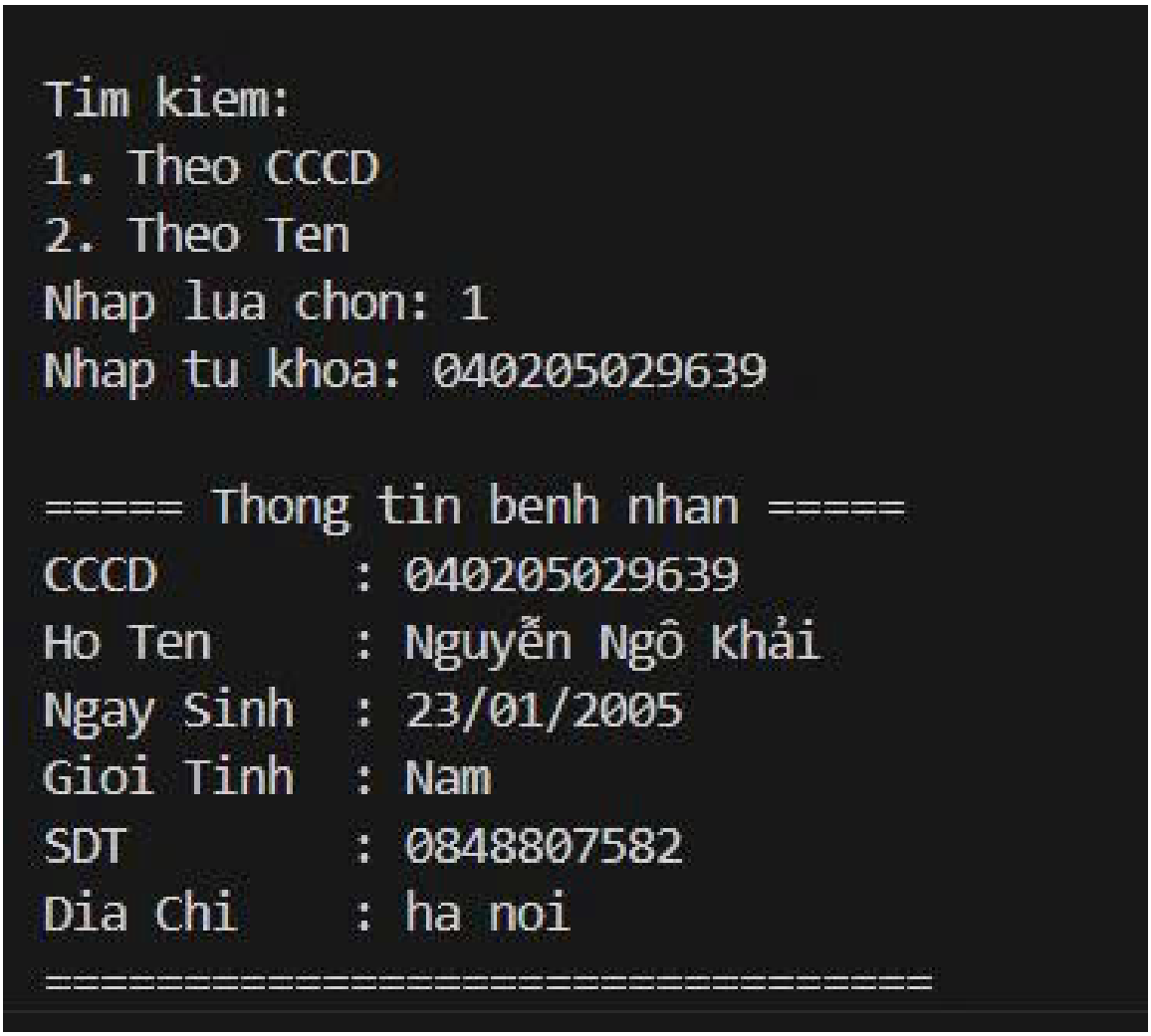
TEST 9: Chọn chức năng

- **INPUT:** Nhập số "4".
- **OUTPUT :** Hiển thị các lựa chọn tìm kiếm theo CCCD hoặc theo tên.



TEST 10: Chọn phương thức tìm kiếm theo CCCD

- **INPUT:** Nhập số "1" (tìm theo CCCD) và nhập CCCD bệnh nhân cần tìm.
- **OUTPUT :** Hiển thị thông tin bệnh nhân tương ứng.



TEST 11: Chọn phương thức tìm kiếm theo tên

- **INPUT:** Nhập số "2" (tìm theo tên) và nhập tên bệnh nhân cần tìm.
- **OUTPUT :** Hiển thị thông tin bệnh nhân tương ứng.

```
Nhap lua chon: 2
Nhap tu khoa: Anh

Benh nhan chua tu khoa 'Anh':

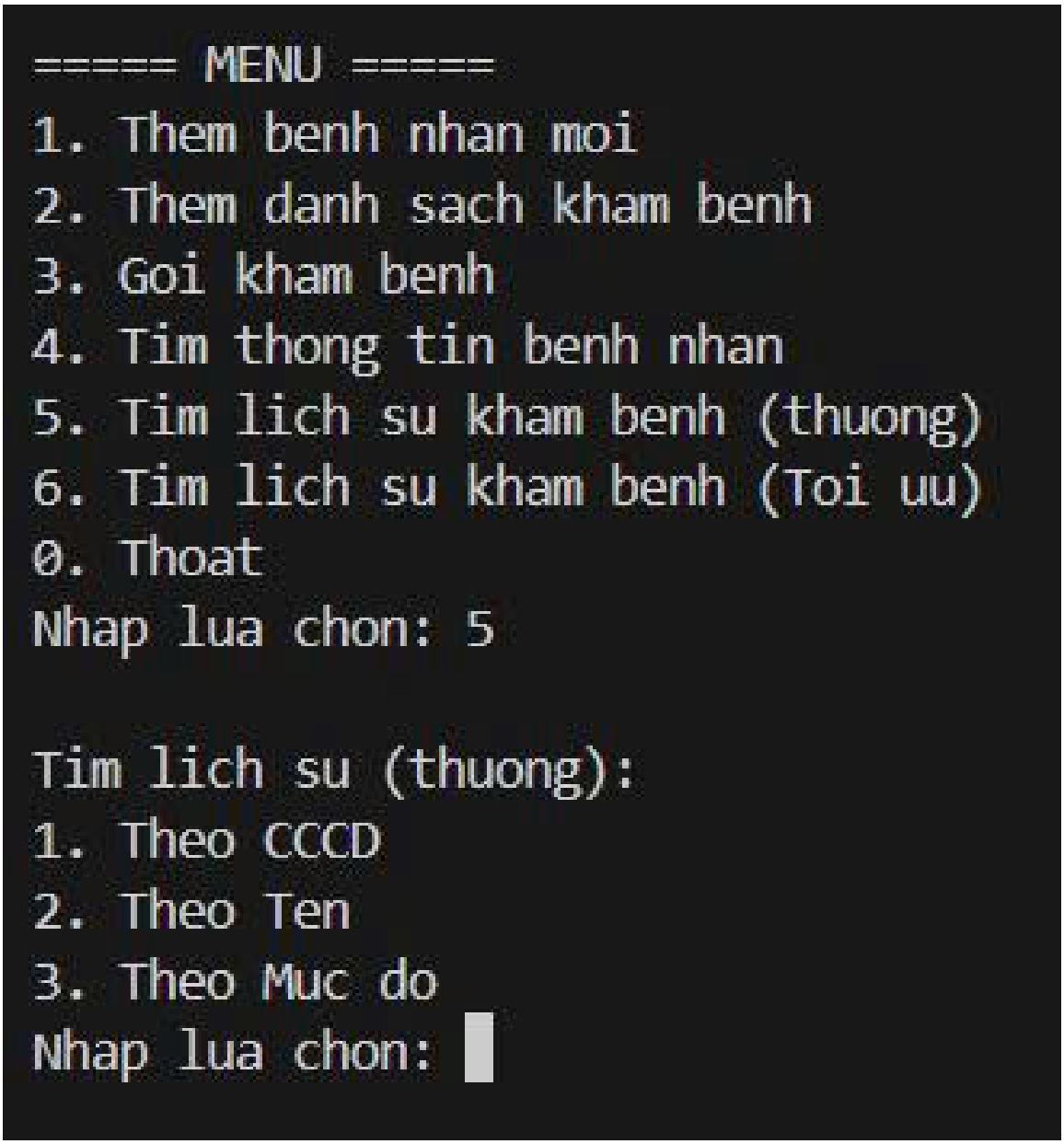
===== Thong tin benh nhan =====
CCCD        : 040205028362
Ho Ten      : Nguyen Huu Nhat Anh
Ngay Sinh   : 30/01/2005
Gioi Tinh   : Nu
SDT         : 0947264859
Dia Chi     : 120 Yen Lang
================================

===== Thong tin benh nhan =====
CCCD        : 040206048574
Ho Ten      : nhat anh
Ngay Sinh   : 24/06/2005
Gioi Tinh   : Nam
SDT         : 0856478593
Dia Chi     : an lieng
================================
```

4.6 Chức năng tìm kiếm lịch sử khám bệnh

TEST 12: Chọn chức năng

- **INPUT:** Nhập số "5".
- **OUTPUT :** Hiển thị các lựa chọn tìm kiếm theo CCCD, theo tên hoặc theo mức độ.



TEST 13: Chọn phương thức tìm kiếm lịch sử theo CCCD

- **INPUT:** Nhập số "1" (tìm theo CCCD) và nhập CCCD bệnh nhân cần tìm.
- **OUTPUT :** Hiển thị lịch sử khám của bệnh nhân tương ứng.

```
Tim lich su (thuong):
1. Theo CCCD
2. Theo Ten
3. Theo Muc do
Nhap lua chon: 1
Nhap tu khoa: 040205029639

-- Lich su kham --
CCCD     : 040205029639
Ten      : Nguyễn Ngô Khải
Ngay sinh: 23/01/2005
Muc do   : 1
Ngay kham: 08/12/2025
Mo ta    : da khoi benh

-- Lich su kham --
CCCD     : 040205029639
Ten      : Nguyễn Ngô Khải
Ngay sinh: 23/01/2005
Muc do   : 1
Ngay kham: 01/04/2026
Mo ta    : TGDK:05/01/2026 09:12:50; Cap cuu noi soi da day;
```

TEST 14: Chọn phương thức tìm kiếm lịch sử theo tên

- **INPUT:** Nhập số "2" (tìm theo tên) và nhập tên bệnh nhân cần tìm.
- **OUTPUT :** Hiển thị lịch sử khám của bệnh nhân tương ứng.

```
Nhap lua chon: 6

Tim kiem lich su kham (Toi uu):
1. Theo CCCD
2. Theo Ten
3. Theo Muc do
Chon: 2
Nhap tu khoa ten: Anh

-- Lich su kham --
CCCD     : 040206048574
Ten      : nhat anh
Ngay sinh: 24/06/2005
Muc do   : 1
Ngay kham:
Mo ta    :

-- Lich su kham --
CCCD     : 040206048574
Ten      : nhat anh
Ngay sinh: 24/06/2005
Muc do   : 1
Ngay kham:
Mo ta    : TGDK:02/01/2026 21:11:48 | ffaaff
```

TEST 15: Chọn phương thức tìm kiếm lịch sử theo mức độ

- **INPUT:** Nhập số "3" (tìm theo mức độ) và nhập mức độ cần tìm.
- **OUTPUT :** Hiển thị lịch sử khám của các bệnh nhân có mức độ tương ứng.

```
Chon: 3
Nhap muc do (1-3): 1

-- Lich su kham --
CCCD      : 040205029639
Ten       : Nguyễn Ngô Khải
Ngay sinh: 23/01/2005
Muc do    : 1
Ngay kham: 08/12/2025
Mo ta     : da khoi benh

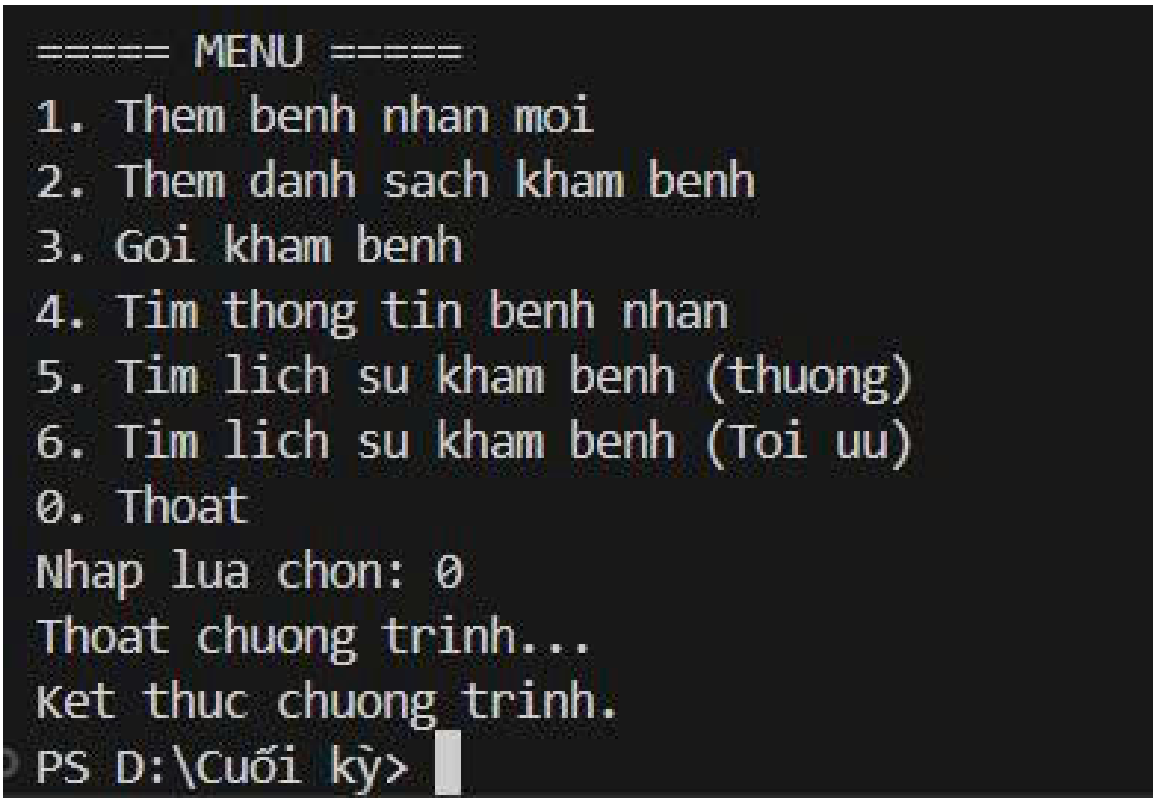
-- Lich su kham --
CCCD      : 040205028537
Ten       : Nguyen Huy Hiep
Ngay sinh: 20/02/2006
Muc do    : 1
Ngay kham: 23/05/2025
Mo ta     : chet

-- Lich su kham --
CCCD      : 040205027483
Ten       : khai
Ngay sinh: 20/01/2005
Muc do    : 1
Ngay kham: 24/05/052026
Mo ta     : đã khỏi
```

4.7 Thoát chương trình và đóng file

TEST 16: Chọn chức năng thoát

- **INPUT:** Nhập số "0".
- **OUTPUT :** Chương trình kết thúc, đóng file dữ liệu và thoát khỏi hệ thống.


===== MENU =====
1. Them benh nhan moi
2. Them danh sach kham benh
3. Goi kham benh
4. Tim thong tin benh nhan
5. Tim lich su kham benh (thuong)
6. Tim lich su kham benh (Toi uu)
0. Thoat
Nhap lua chon: 0
Thoat chuong trinh...
Ket thuc chuong trinh.
PS D:\Cuối kỳ>

Phụ lục

A. Mã nguồn main.py

```
# main.py
from benhnhan_hash import BenhNhanHash
from trie_ds import Trie
from benhnhan import themBenhNhan, timBenhNhan
from kham import KhamManager
from lichsu2 import LichSuManager
from menu import MenuItem, inMenu, xuLyLuaChon


def menuTimLichSuToiUu(lichsu_mgr: LichSuManager) -> None:
    print("\nTim kiem lich su kham (Toi uu):")
    print("1. Theo CCCD")
    print("2. Theo Ten")
    print("3. Theo Muc do")
    try:
        luaChon = int(input("Chon: ").strip())
    except ValueError:
        print(" Lua chon khong hop le.")
        return

    if luaChon == 1:
        cccd = input("Nhap CCCD: ").strip()
        lichsu_mgr.timTheoCCCD(cccd)
    elif luaChon == 2:
        tuKhoa = input("Nhap tu khoa ten: ").strip()
        lichsu_mgr.timTheoTen(tuKhoa)
    elif luaChon == 3:
        try:
            mucDo = int(input("Nhap muc do (1-3): ").strip())
        except ValueError:
            print(" Muc do khong hop le.")
            return
        lichsu_mgr.timTheoMucDo(mucDo)
    else:
        print(" Lua chon khong hop le.")


def main():
    print("Bat dau chuong trinh...")

    # Load du lieu benh nhan (HashTable) :contentReference[oaicite:11]{index
        =11}
    ht = BenhNhanHash.load_from_file("BENHNHAN.TXT")
    if not ht.table:
        print("Khong the tai (hoac khong co) du lieu benh nhan.")
    else:
        print("Tai du lieu benh nhan thanh cong.")
```

```
47
48      # Khoi tao trie & nap benh nhan vao trie
49      trie_bn = Trie()
50      for bn in ht.values():
51          trie_bn.insert_name(bn.hoTen, bn)
52
53      # Khoi tao & load lich su toi uu
54      lichsu_mgr = LichSuManager()
55      lichsu_mgr.load_from_file("LICHSUKHAMBENH.TXT")
56
57      # Manager kham benh (heap)
58      kham_mgr = KhamManager(ht, lichsu_mgr, history_file="LICHSUKHAMBENH.TXT")
59
60      # Cn bc cc hm  ph hp ch k action() khng tham s
61      def action_themBenhNhan():
62          themBenhNhan(ht, trie_bn, "BENHNHAN.TXT")
63
64      def action_themKhamBenh():
65          kham_mgr.themKhamBenh()
66
67      def action_goiKhamBenh():
68          kham_mgr.goiKhamBenh()
69
70      def action_timBenhNhan():
71          timBenhNhan(ht, trie_bn)
72
73      def action_timLichSuThuong():
74          from lichsu2 import LichSuManager
75          LichSuManager.timLichSuKham_thuong("LICHSUKHAMBENH.TXT")
76
77      def action_timLichSuToiUu():
78          menuTimLichSuToiUu(lichsu_mgr)
79
80      menu = [
81          MenuItem(1, "Them benh nhan moi", action_themBenhNhan),
82          MenuItem(2, "Them danh sach kham benh", action_themKhamBenh),
83          MenuItem(3, "Goi kham benh", action_goiKhamBenh),
84          MenuItem(4, "Tim thong tin benh nhan", action_timBenhNhan),
85          MenuItem(5, "Tim lich su kham benh (thuong)", action_timLichSuThuong),
86          MenuItem(6, "Tim lich su kham benh (Toi uu)", action_timLichSuToiUu),
87          MenuItem(0, "Thoat", None),
88      ]
89
90      while True:
91          inMenu(menu)
92          try:
93              choice = int(input("Nhap lua chon: ").strip())
94          except ValueError:
95              print("Nhap sai! Vui long nhap mot so.")
96              continue
97
```

```
        if choice == 0:
            xuLyLuaChon(menu, 0)
            break

        xuLyLuaChon(menu, choice)

    print("Ket thuc chuong trinh.")


if __name__ == "__main__":
    main()
```

B. Mã nguồn menu.py

```
# menu.py
from dataclasses import dataclass
from typing import Callable, Optional


@dataclass
class MenuItem:
    id: int
    text: str
    action: Optional[Callable[[], None]]


def inMenu(menu: list[MenuItem]) -> None:
    print("\n===== MENU =====")
    for item in menu:
        print(f"{item.id}. {item.text}")


def xuLyLuaChon(menu: list[MenuItem], id: int) -> None:
    for item in menu:
        if item.id == id:
            if item.action is not None:
                item.action()
            else:
                print("Thoat chuong trinh...")
            return
    print(" Lua chon khong hop le.")
```

C. Mã nguồn benhnhan.py

```
# benhnhan.py
from dataclasses import dataclass


@dataclass
```

```python
class BenhNhan:
    cccd: str
    hoTen: str
    ngaySinh: str
    gioiTinh: int # 1: Nam, 0: Nu
    sdt: str
    diaChi: str


def _kiemTraCCCD(cccd: str) -> bool:
    return len(cccd) == 12 and cccd.isdigit()


def _kiemTraSDT(sdt: str) -> bool:
    return len(sdt) == 10 and sdt.isdigit()


def _kiemTraTen(ten: str) -> bool:
    ten = ten.strip()
    return 4 <= len(ten) <= 50


def _kiemTraDiaChi(dc: str) -> bool:
    dc = dc.strip()
    return 3 <= len(dc) <= 50


def _kiemTraNgaySinh(ns: str) -> bool:
    if len(ns) != 10:
        return False
    if ns[2] != "/" or ns[5] != "/":
        return False
    try:
        d, m, y = map(int, ns.split("/"))
        return 1 <= d <= 31 and 1 <= m <= 12 and 1900 <= y <= 2100
    except Exception:
        return False


def inThongTinBenhNhan(bn: BenhNhan) -> None:
    gioi_tinh_str = "Nam" if bn.gioiTinh == 1 else "Nu"
    print("\n===== Thong tin benh nhan =====")
    print(f"CCCD : {bn.cccd}")
    print(f"Ho Ten : {bn.hoTen}")
    print(f"Ngay Sinh : {bn.ngaySinh}")
    print(f"Gioi Tinh : {gioi_tinh_str}")
    print(f"SDT : {bn.sdt}")
    print(f"Dia Chi : {bn.diaChi}")
    print("================================")


```

```python
def themBenhNhan(ht, trie, filename: str = "BENHNHAN.TXT") -> None:
    """
    ht : i tng BenhNhanHash (nhng khng cn import  y)
    trie : i tng Trie (khng cn import  y)
    """
    print("\n=== Them benh nhan moi ===")

    # CCCD
    while True:
        cccd = input("Nhap CCCD (12 so): ").strip()
        if not _kiemTraCCCD(cccd):
            print(" CCCD khong hop le, nhap lai.")
            continue
        if ht.search(cccd) is not None:
            print(" CCCD da ton tai, nhap so khac.")
            continue
        break

    # H tn
    while True:
        hoTen = input("Nhap ho ten (4-50 ky tu): ").strip()
        if _kiemTraTen(hoTen):
            break
        print(" Ho ten khong hop le, nhap lai.")

    # Ngy sinh
    while True:
        ngaySinh = input("Nhap ngay sinh (DD/MM/YYYY): ").strip()
        if _kiemTraNgaySinh(ngaySinh):
            break
        print(" Ngay sinh khong hop le, nhap lai.")

    # Gii tnh
    while True:
        try:
            gioiTinh = int(input("Nhap gioi tinh (1: Nam, 0: Nu): ").strip())
        except ValueError:
            print(" Vui long nhap 0 hoac 1.")
            continue
        if gioiTinh in (0, 1):
            break
        print(" Gioi tinh khong hop le, nhap lai.")

    # ST
    while True:
        sdt = input("Nhap so dien thoai (10 so): ").strip()
        if _kiemTraSDT(sdt):
            break
        print(" So dien thoai khong hop le, nhap lai.")

    # a ch
```

```
    while True:
        diaChi = input("Nhap dia chi (3-50 ky tu): ").strip()
        if _kiemTraDiaChi(diaChi):
            break
        print(" Dia chi khong hop le, nhap lai.")

    bn = BenhNhan(cccd, hoTen, ngaySinh, gioiTinh, sdt, diaChi)

    # Ghi file
    with open(filename, "a", encoding="utf-8") as f:
        f.write(f"{bn.cccd}|{bn.hoTen}|{bn.ngaySinh}|{bn.gioiTinh}|{bn.sdt}|{bn.
            diaChi}\n")

    # Cp nht hash & trie (2 i tng ny c truyn vo t main)
    ht.insert(bn)
    trie.insert_name(bn.hoTen, bn)

    print(" Da them benh nhan thanh cong!")


def timBenhNhan(ht, trie) -> None:
    print("\nTim kiem:")
    print("1. Theo CCCD")
    print("2. Theo Ten")
    try:
        luaChon = int(input("Nhap lua chon: ").strip())
    except ValueError:
        print(" Lua chon khong hop le.")
        return

    tuKhoa = input("Nhap tu khoa: ").strip()

    if luaChon == 1:
        bn = ht.search(tuKhoa)
        if bn:
            inThongTinBenhNhan(bn)
        else:
            print(" Khong tim thay benh nhan!")
    elif luaChon == 2:
        results = trie.search_keyword(tuKhoa)
        if not results:
            print(" Khong tim thay benh nhan co tu khoa nay.")
            return
        print(f"\nBenh nhan chua tu khoa '{tuKhoa}':")
        for bn in results:
            inThongTinBenhNhan(bn)
    else:
        print(" Lua chon khong hop le.")
```

D. Mã nguồn benhnhan_hash.py

```python
from benhnhan import BenhNhan


class BenhNhanHash:
    """
    Bng bm qun l bnh nhn theo CCCD.
    Thay th cho HashTable trong C.
    """

    def __init__(self):
        # key: cccd (str) -> value: BenhNhan
        self.table: dict[str, BenhNhan] = {}

    def insert(self, bn: BenhNhan) -> None:
        """Thm 1 bnh nhn vo hash."""
        self.table[bn.cccd] = bn

    def search(self, cccd: str) -> BenhNhan | None:
        """Tm bnh nhn theo CCCD. Khng thy th tr v None."""
        return self.table.get(cccd)

    def values(self):
        """Tr v iterator tt c bnh nhn (dng  np vo Trie)."""
        return self.table.values()

    @staticmethod
    def load_from_file(filename: str) -> "BenhNhanHash":
        """c BENHNHAN.TXT v a vo hash."""
        ht = BenhNhanHash()
        try:
            with open(filename, "r", encoding="utf-8") as f:
                for line in f:
                    line = line.strip()
                    if not line:
                        continue
                    parts = line.split("|")
                    if len(parts) != 6:
                        continue

                    cccd, hoTen, ngaySinh, gioiTinh_str, sdt, diaChi = parts
                    try:
                        gioiTinh = int(gioiTinh_str)
                    except ValueError:
                        gioiTinh = 1 # mc nh Nam

                    bn = BenhNhan(cccd, hoTen, ngaySinh, gioiTinh, sdt, diaChi)
                    ht.insert(bn)
        except FileNotFoundError:
            # Khng c file vn OK, tr v hash rng
```

```
            pass

        return ht
```

E. Mã nguồn kham.py

```
# kham.py
from dataclasses import dataclass
import heapq
from datetime import datetime

from benhnhan import BenhNhan, inThongTinBenhNhan
from lichsu2 import LichSuKham


@dataclass
class PhieuKham:
    bn: BenhNhan
    mucUuTien: int # 1: cap cuu, 2: uu tien, 3: thuong
    moTa: str # mo ta/loai benh luc dang ky (<=50)
    sttKham: int # STT (chi dung cho muc 3)
    arrival: int # thu tu den (pha hoa FIFO cho muc 1/2)
    thoiGianDangKy: str # "DD/MM/YYYY HH:MM:SS"


class KhamManager:
    def __init__(self, ht, lichsu_manager, history_file: str = "LICHSUKHAMBENH.
        TXT"):
        self.heap: list[tuple[tuple[int, int, int, int], PhieuKham]] = []

        self.arrival_counter = 0
        self.stt_thuong_counter = 0
        self.so_thu_tu_duoc_kham = 0

        self.ht = ht
        self.lichsu = lichsu_manager
        self.history_file = history_file

    def _make_key(self, pk: PhieuKham) -> tuple[int, int, int, int]:
        prio_rank = {1: 0, 2: 1, 3: 2}.get(pk.mucUuTien, 2)

        if pk.mucUuTien in (1, 2):
            group = 0
            third = pk.arrival
        else:
            group = 1
            third = pk.sttKham

        return (prio_rank, group, third, pk.arrival)

```

```
    def _tg_hms(self, tg: str) -> str:
        if tg and len(tg) >= 8:
            return tg[-8:]
        return ""

    @staticmethod
    def _safe_text(s: str) -> str:
        # trnh ph format cccd|hoTen|...|moTa
        return (s or "").replace("|", "/")

    def _compose_history_note(self, pk: PhieuKham, ketqua: str) -> str:
        """
        Gi ng format file lch s 6 trng:
        cccd|hoTen|ngaySinh|mucUuTien|ngayKham|moTa

        => tuyt i KHNG dng k t '|' trong moTa
        """
        parts = []
        if pk.mucUuTien == 1:
            parts.append(f"TGDK:{pk.thoiGianDangKy}")
        if pk.moTa:
            parts.append(pk.moTa)
        if ketqua:
            parts.append(ketqua)

        # Dng '; ' thay v ' | '  khng to thm ct khi split("|")
        return "; ".join(parts)[:50]

    def themKhamBenh(self) -> None:
        cccd = input("Nhap CCCD benh nhan: ").strip()
        node = self.ht.search(cccd)
        if not node:
            print(" Khong tim thay benh nhan!")
            return

        inThongTinBenhNhan(node)

        while True:
            try:
                muc = int(input("Nhap muc do uu tien (1: cap cuu, 2: uu tien, 3:
                    thuong): ").strip())
            except ValueError:
                print(" Muc do khong hop le.")
                continue
            if muc not in (1, 2, 3):
                print(" Muc do khong hop le.")
                continue
            break

        moTa = self._safe_text(input("Nhap mo ta/loai benh (<50 ky tu): ").strip
            ())[:50]
```

```
93
94          tgdk = datetime.now().strftime("%d/%m/%Y %H:%M:%S")
95
96          self.arrival_counter += 1
97          arrival = self.arrival_counter
98
99          if muc == 3:
100             self.stt_thuong_counter += 1
101             stt = self.stt_thuong_counter
102         else:
103             stt = 0
104
105         pk = PhieuKham(node, muc, moTa, stt, arrival, tgdk)
106         heapq.heappush(self.heap, (self._make_key(pk), pk))
107
108         print(" Da them vao danh sach cho kham.")
109         if muc == 1:
110             print(f" (CAP CUU) Thoi gian dang ky: {tgdk}")
111
112     def goiKhamBenh(self) -> None:
113         if not self.heap:
114             print(" Khong co benh nhan trong danh sach cho!")
115             return
116
117         waiting = sorted(self.heap, key=lambda x: x[0])
118
119         print("\nDanh sach cho kham:")
120         for idx, (key, pk) in enumerate(waiting, start=1):
121             tg_show = self._tg_hms(pk.thoiGianDangKy)
122             tg_prefix = f"[{tg_show}] " if tg_show else ""
123
124             if pk.mucUuTien == 3:
125                 print(f"#{idx} {tg_prefix}(THUONG - STT {pk.sttKham}) - {pk.bn.
                        hoTen} - {pk.bn.ngaySinh}")
126             else:
127                 print(f"#{idx} {tg_prefix}(UU TIEN {pk.mucUuTien}) - {pk.bn.
                        hoTen} - {pk.bn.ngaySinh}")
128
129         input("\nNhan Enter de goi benh nhan dau tien... ")
130
131         _, goi = heapq.heappop(self.heap)
132         self.so_thu_tu_duoc_kham += 1
133
134         print(f"\n>>> Dang goi benh nhan #{self.so_thu_tu_duoc_kham}:")
135         inThongTinBenhNhan(goi.bn)
136
137         if goi.mucUuTien == 1:
138             print(f"*** CAP CUU - Thoi gian dang ky: {goi.thoiGianDangKy} ***")
139
140         input("\nKhi benh nhan kham xong an Enter... ")
141
```

```
        ngayKham = input("Nhap ngay kham (DD/MM/YYYY): ").strip()
        moTaKham = self._safe_text(input("Nhap ket qua kham benh (<50): ").strip
            ())[:50]

        moTaGhi = self._compose_history_note(goi, moTaKham)

        with open(self.history_file, "a", encoding="utf-8") as fp:
            fp.write(
                f"{goi.bn.cccd}|{goi.bn.hoTen}|{goi.bn.ngaySinh}|"
                f"{goi.mucUuTien}|{ngayKham}|{moTaGhi}\n"
            )

        ls = LichSuKham(
            cccd=goi.bn.cccd,
            hoTen=goi.bn.hoTen,
            ngaySinh=goi.bn.ngaySinh,
            mucDo=goi.mucUuTien,
            ngayKham=ngayKham,
            moTa=moTaGhi,
        )
        self.lichsu.add_record(ls)

        print(" Da luu vao lich su kham benh.")
```

F. Mã nguồn `lichsu2.py`

```
# lichsu2.py
from dataclasses import dataclass
from trie_ds import Trie


@dataclass
class LichSuKham:
    cccd: str
    hoTen: str
    ngaySinh: str
    mucDo: int # 1,2,3
    ngayKham: str
    moTa: str


class LichSuManager:
    """
    - Hash theo CCCD
    - Trie theo tn
    - Array of lists theo mucDo
    """

    def __init__(self):
        self.by_cccd: dict[str, list[LichSuKham]] = {}
```

```
        self.trie = Trie()
        self.by_mucdo: list[list[LichSuKham]] = [[], [], []] # index 0->muc1,
            1->muc2, 2->muc3

    def add_record(self, ls: LichSuKham) -> None:
        self.by_cccd.setdefault(ls.cccd, []).append(ls)
        self.trie.insert_name(ls.hoTen, ls)
        if 1 <= ls.mucDo <= 3:
            self.by_mucdo[ls.mucDo - 1].append(ls)

    def load_from_file(self, filename: str) -> None:
        """
        c file lch s theo format 6 trng:
        cccd|hoTen|ngaySinh|mucDo|ngayKham|moTa

        SA: dng split("|", 5)  trnh li nu moTa c cha k t '|'
        """
        try:
            with open(filename, "r", encoding="utf-8") as f:
                for line in f:
                    line = line.strip()
                    if not line:
                        continue

                    parts = line.split("|", 5) # <= CHT: ch tch ti a 6 trng
                    if len(parts) < 6:
                        continue

                    cccd, hoTen, ngaySinh, mucDo_str, ngayKham, moTa = parts
                    try:
                        mucDo = int(mucDo_str)
                    except ValueError:
                        mucDo = 3

                    ls = LichSuKham(cccd, hoTen, ngaySinh, mucDo, ngayKham, moTa)
                    self.add_record(ls)
        except FileNotFoundError:
            # Khng c lch s =>  trng
            pass

    def timTheoCCCD(self, cccd: str) -> None:
        ds = self.by_cccd.get(cccd)
        if not ds:
            print(" Khong co lich su voi CCCD nay.")
            return
        for ls in ds:
            self._in_lich_su(ls)

    def timTheoTen(self, tuKhoa: str) -> None:
        results = self.trie.search_keyword(tuKhoa)
        if not results:
```

```
75            print(" Khong co lich su voi tu khoa ten nay.")
76            return
77        for ls in results:
78            self._in_lich_su(ls)
79
80    def timTheoMucDo(self, mucDo: int) -> None:
81        if not (1 <= mucDo <= 3):
82            print(" Muc do khong phu hop (1-3).")
83            return
84        ds = self.by_mucdo[mucDo - 1]
85        if not ds:
86            print(" Khong co lich su voi muc do nay.")
87            return
88        for ls in ds:
89            self._in_lich_su(ls)
90
91    @staticmethod
92    def _in_lich_su(ls: LichSuKham) -> None:
93        print("\n-- Lich su kham --")
94        print(f"CCCD : {ls.cccd}")
95        print(f"Ten : {ls.hoTen}")
96        print(f"Ngay sinh: {ls.ngaySinh}")
97        print(f"Muc do : {ls.mucDo}")
98        print(f"Ngay kham: {ls.ngayKham}")
99        print(f"Mo ta : {ls.moTa}")
100
101   # Bn "thng"  c file mi ln
102   @staticmethod
103   def timLichSuKham_thuong(filename: str = "LICHSUKHAMBENH.TXT") -> None:
104       try:
105           fp = open(filename, "r", encoding="utf-8")
106       except FileNotFoundError:
107           print(f"Khong the mo file {filename}")
108           return
109
110       print("\nTim lich su (thuong):")
111       print("1. Theo CCCD")
112       print("2. Theo Ten")
113       print("3. Theo Muc do")
114       try:
115           luaChon = int(input("Nhap lua chon: ").strip())
116       except ValueError:
117           print(" Lua chon khong hop le.")
118           fp.close()
119           return
120
121       tuKhoa = ""
122       mucDoTim = 0
123       if luaChon == 3:
124           try:
125               mucDoTim = int(input("Nhap muc do (1/2/3): ").strip())
```

```
            except ValueError:
                print(" Muc do khong hop le.")
                fp.close()
                return
        else:
            tuKhoa = input("Nhap tu khoa: ").strip()

        timThay = False
        for line in fp:
            line = line.strip()
            if not line:
                continue

            # <= CHT: ch tch ti a 6 trng, phn d dn vo moTa
            parts = line.split("|", 5)
            if len(parts) < 6:
                continue

            cccd, hoTen, ngaySinh, mucDo_str, ngayKham, moTa = parts
            try:
                mucDo = int(mucDo_str)
            except ValueError:
                mucDo = 3

            match = False
            if luaChon == 1 and cccd == tuKhoa:
                match = True
            elif luaChon == 2 and tuKhoa in hoTen:
                match = True
            elif luaChon == 3 and mucDo == mucDoTim:
                match = True

            if match:
                timThay = True
                print("\n-- Lich su kham --")
                print(f"CCCD : {cccd}")
                print(f"Ten : {hoTen}")
                print(f"Ngay sinh: {ngaySinh}")
                print(f"Muc do : {mucDo}")
                print(f"Ngay kham: {ngayKham}")
                print(f"Mo ta : {moTa}")

        if not timThay:
            print("Khong tim thay lich su kham phu hop.")

        fp.close()
```

G. Mã nguồn `trie_ds.py`

```
# trie_ds.py
```

```python
from collections import defaultdict


class TrieNode:
    def __init__(self):
        self.children: dict[str, "TrieNode"] = {}
        self.records: list[object] = [] # lu BenhNhan hoc LichSuKham


class Trie:
    def __init__(self):
        self.root = TrieNode()

    def _insert_word(self, word: str, record: object) -> None:
        p = self.root
        for ch in word.lower():
            if ch not in p.children:
                p.children[ch] = TrieNode()
            p = p.children[ch]
        p.records.append(record)

    def insert_name(self, name: str, record: object) -> None:
        # ging chenBenhNhanVaoTrie: tch theo du cch :contentReference[oaicite
            :6]{index=6}
        for token in name.strip().split():
            if token:
                self._insert_word(token, record)

    def search_keyword(self, keyword: str) -> list[object]:
        p = self.root
        for ch in keyword.lower():
            if ch not in p.children:
                return []
            p = p.children[ch]
        # c th loi trng (record xut hin nhiu ln nu nhiu t trng)
        seen = set()
        results = []
        for rec in p.records:
            if id(rec) not in seen:
                seen.add(id(rec))
                results.append(rec)
        return results
```

Tài liệu tham khảo

1. Đỗ Xuân Lôi, *Cấu trúc dữ liệu và giải thuật*, Đại học Quốc Gia Hà Nội.
2. Nguyễn Đức Nghĩa, *Cấu trúc dữ liệu và giải thuật*, Đại học Bách Khoa Hà Nội.